



Application - Intégrer les technologies d'Intelligence Artificielle dans une solution logicielle

Table des matières

I - Exercice : Partie 1 : Cas pratique EduFrance	3
II - Exercice : Partie 2 : Mises en situation spécifiques	4
Solutions des exercices	5

I Exercice : Partie 1 : Cas pratique EduFrance

Vous êtes développeur full-stack chez EduFrance, une entreprise française qui développe des logiciels pour les organismes de formation.

L'entreprise souhaite développer une application web qui aidera les formateurs et les apprenants à optimiser leur parcours grâce à l'IA.

L'application doit intégrer deux fonctionnalités principales :

1. Un chatbot multimodal (texte, voix, image) qui répond aux questions de l'apprenant sur un sujet donné.
2. Une fonctionnalité d'upsell : recommandation de cours complémentaires basées sur le panier de l'apprenant.

Question 1

[solution n°1 p. 5]

Rédigez les user stories puis l'analyse technique associée à chaque fonctionnalité :

Ce qui est attendu :

- Décrivez le cycle de vie de chaque fonctionnalité (de la collecte de données à la mise à jour continue),
- Évaluez les technologies d'IA les plus adaptées à chaque fonctionnalité,
- Justifiez vos choix en tenant compte des contraintes éthiques et de la conformité RGPD,
- Présentez une première version du projet ainsi qu'une V2 plus avancée,
- Intégrez les bonnes pratiques de développement.

Question 2

[solution n°2 p. 7]

Proposez une architecture technique backend intégrant les deux fonctionnalités.

Ce qui est attendu :

- La stratégie modulaire employée
- Le rôle de chaque dossier et fichiers par fonctionnalités

Question 3

[solution n°3 p. 8]

Proposez l'endpoint ainsi que la fonction "get_upsell".

II Exercice : Partie 2 : Mises en situation spécifiques

Scénario 1 : Intégration d'une nouvelle capacité IA et amélioration du chatbot multimodal

EduFrance souhaite ajouter une fonctionnalité de génération de cours personnalisé basé sur les intérêts de l'apprenant ainsi qu'un système d'analyse de sentiment pour le chatbot.

Question 1

[solution n°4 p. 8]

Sélectionnez la technologie la plus adaptée pour les deux fonctionnalités supplémentaires.

Question 2

[solution n°5 p. 9]

Créez le prompt de la fonctionnalité de génération de cours personnalisé.

Question 3

[solution n°6 p. 9]

Créez l'endpoint de la fonctionnalité de génération de cours.

Scénario 2 : Optimisation de performances

Le système de génération automatique de cours présente des latences importantes lorsque plusieurs apprenants l'utilisent simultanément.

Question 4

[solution n°7 p. 10]

Vous devez proposer l'implémentation d'une API pour améliorer la latence et les performances globales de la fonctionnalité.

Solutions des exercices

[exercice p. 3] **Solution n°1**

Fonctionnalité 1 : Chatbot Multimodal

User story : en tant qu'apprenant, je peux poser des questions au chatbot sous forme de texte, voix et image afin d'obtenir des recommandations basées sur le cours.

Fonctionnalité texte :

- **Cycle de vie** :
 1. Collecte de données : le chatbot va s'appuyer sur une base de données de cours. Ce dataset servira de base pour répondre aux questions des apprenants.
 2. Entrainement : aucun fine-tuning pour cette fonctionnalité. Nous utiliserons un modèle pré-entraîné (Mistral-7B).
 3. Inférence : lors d'une requête, le chatbot interroge d'abord le dataset. Si aucune correspondance n'est trouvée, l'IA prend le relais.
 4. Validation : nous testerons les deux cas : questions couvertes par le dataset. Et questions hors périmètre (fallback IA).
 5. Réentraînement et mise à jour : pour la V2, possibilité d'intégrer une API externe comme par exemple OpenAI afin d'améliorer les performances du chatbot.
- **Analyse technique** :
 - Backend : FastAPI, modèle d'IA Mistral-7B pour la génération de texte car il s'agit d'un modèle opensource, local et léger,
 - Frontend : React + React Router + Vite + Tailwind.
- **Considérations RGPD et éthiques** :
 - Les informations échangées avec le chatbot ne devront pas être stockées sans l'accord au préalable de l'utilisateur.
 - Si les données sont stockées, il faudra anonymiser les données.
 - Il faudra documenter notre utilisation de l'IA de manière claire.
- **Bonnes pratiques de développement** :
 - Versionnage du code avec Git.
 - Utilisation de DVC pour versionner les modèles IA.
 - Le chatbot sera modulaire, avec un endpoint dédié pour chaque modalité.

Fonctionnalité voix :

User story : en tant qu'apprenant je pose une question à l'oral au chatbot afin d'obtenir des recommandations basées sur le cours.

- **Cycle de vie** :
 1. Collecte de données : la voix de l'apprenant est enregistrée temporairement puis transcrite en texte.
 2. Entrainement : pas de fine tuning pour cette fonctionnalité. Nous allons utiliser un modèle pré entraîné (Vosk).
 3. Inférence : le chatbot analyse la question transcrite et génère une réponse personnalisée en s'appuyant sur les connaissances ducours.

4. Validation : nous vérifions la réponse par rapport aux données disponibles dans le dataset et vérifions que les questions couvertes par le fallback IA sont fiables.
5. Réentraînement et mise à jour : pour une V2 nous pouvons utiliser une API pour accélérer le traitement des questions. Des enquêtes utilisateurs permettront de mesurer l'efficacité.

- **Analyse technique :**

- Backend : modèle IA Vosk car il s'agit d'un modèle léger et opensource. Seule limite ici, nous commencerons par tester notre endpoint avec un modèle Vosk en anglais puis nous pourrons basculer vers un modèle IA français après avoir validé la fonctionnalité.
- Frontend : React + React Router + Vite + Tailwind.

- **Considérations RGPD et éthiques :**

- Ne pas stocker la voix de l'utilisateur sans son consentement explicite.
- Si la voix est stockée, anonymiser les données.
- Documenter l'usage de l'IA.

- **Bonnes pratiques de développement :**

- Versionnage de code avec Git,
- Utilisation de DVC pour versionner les modèles IA.

Fonctionnalité Image :

User Story : en tant qu'apprenant je peux envoyer la photo d'un diagramme ou de tout autre élément visuel au chatbot afin d'obtenir une explication claire.

- **Cycle de vie :**

1. Collecte de données : les photos envoyées par les apprenants sont traitées comme données d'entrée.
2. Entraînement : pas de fine-tuning. Nous utiliserons un modèle pré-entraîné local et opensource (Llama-vision).
3. Inférence : le pipeline de vision Llama-vision analyse l'image et génère une explication adaptée.
4. Validation : les résultats sont testés avec différents visuels pour vérifier la pertinence et la clarté des réponses.
5. Réentraînement et mise à jour : recours à une API spécialisée en V2 pour améliorer les performances de la fonctionnalité.

- **Analyse technique :**

- Backend : comme modèle IA nous allons utiliser llama-vision,
- Frontend : React + React Router + Vite + Tailwind.

- **Considérations RGPD et éthiques :**

- Ne pas stocker les images de l'utilisateur sans consentement explicite.
- Si l'image est stockée, anonymiser les données.
- Documenter l'utilisation de l'IA pour cette fonctionnalité.

- **Bonnes pratiques de développement :**

- Versionning Git pour les fichiers légers et le code,
- Utilisation de DVC pour versionner les modèles IA.

Fonctionnalité 2 : Upsell

User story : en tant qu'apprenant, lorsque j'ajoute une formation à mon panier, le système me propose automatiquement une formation complémentaire.

- **Cycle de vie** :

1. Collecte de données : récupération de l'historique d'achats et des parcours de formation.
2. Entrainement : pas de fine tuning, utilisation de Mistral-7B avec prompts adaptés.
3. Inférence : lorsqu'une formation est ajoutée au panier, le modèle IA génère une suggestion contextuelle.
4. Validation : mesurer le taux de clic, d'ajouts au panier et taux de conversion pour mesurer la pertinence de la fonctionnalité.
5. Réentraînement et mise à jour : envisager l'utilisation d'une API (OpenAI par exemple) pour affiner la qualité des recommandations.

- **Analyse technique** :

- **Backend** : modèle IA Mistral-7B, prompts versionnés,
- **Frontend** : React + React Router + Vite + Tailwind. Ajout d'une route « Upsell ».

- **Bonnes pratiques de développement** :

- Versionner le code avec Git,
- Utilisation de DVC pour versionner les modèles IA.

[exercice p. 3] **Solution n°2**

- **Architecture technique** :

- Mono référentiel avec un endpoint par fonctionnalité.
- Application modulaire organisée par dossiers/fichiers.

- **Dossiers principaux** :

1. backend/api

backend/api/main.py : il s'agit du point d'entrée de notre application. Il contiendra l'ensemble de nos endpoints.

backend/api/schemas.py : il contiendra l'ensemble de nos schémas Pydantic.

2. backend/core.

backend/core/services.py : logique métier des différentes fonctionnalités.

backend/core/voice_client.py : logique "voix" du chatbot

backend/core/vision_client.py : logique "image" du chatbot

backend/core/prompt_manager.py : centralisation des prompts

backend/core/prompts : contiendra nos différents prompts pour la

Fonctionnalité upsell. Ceux-ci seront versionnés sous la forme upsell_prompt_v.1.0.txt pour plus de lisibilité et faciliter le testing.

3. backend/models : contient les modèles IA

4. backend/requirements.txt : fige les dépendances et outils

[exercice p. 3] **Solution n°3**

Étape 1 : créer l'endpoint /upsell dans backend/api/main.py

Étape 2 : dans le fichier main.py : importer les modèles Pydantic UpsellResponse et UpsellRequest

Étape 3 : dans le fichier main.py : importer la fonction get_upsell

Étape 4 : création de l'endpoint /upsell :

```
1 @app.post("/upsell", response_model=UpsellResponse)
2 async def upsell(request: UpsellRequest):
3     result = get_upsell(request.cart_items)
4     return UpsellResponse(**result)
```

```
@app.post("/upsell", response_model=UpsellResponse)
async def upsell(request: UpsellRequest):
    result = get_upsell(request.cart_items)
    return UpsellResponse(**result)
```

Étape 5 : création de la fonction get_upsell dans : backend/core/services.py

```
1 def get_upsell(cart_items: list[str]) -> dict:
2     prompt_template = load_prompt(PROMPT_UPSELL)
3     prompt = prompt_template.replace("{cart_items}", ", ".join(cart_items))
4     result = get_llm()(prompt, max_tokens=80)
5     return {"suggestion": result["choices"][0]["text"].stri()}
6
```

```
def get_upsell(cart_items: list[str]) -> dict:
    prompt_template = load_prompt(PROMPT_UPSELL)
    prompt = prompt_template.replace("{cart_items}", ", ".join(cart_items))
    result = get_llm()(prompt, max_tokens=80)
    return {"suggestion": result["choices"][0]["text"].strip() }
```

[exercice p. 4] **Solution n°4**

- **Fonctionnalité génération de cours personnalisé :**

- LLM : Mistral-7B,
- Pourquoi : modèle open-source, léger, exécution locale donc plus de confidentialité, coûts maîtrisés. V2 : basculer vers API si besoin d'amélioration des performances (qualité des réponses, latence, etc.).
- Approche : prompts versionnés (pas de fine-tuning en V1).
- Frontend : ajout d'une route /cours.

- **Fonctionnalité amélioration du chatbot multimodal via Analyse de sentiment :**

Analyse technique : transformers (HuggingFace) avec Pytorch en local.

Règle d'escalade : si label == NEGATIVE et score > 0.75, escalate_to_human = True

[exercice p. 4] **Solution n°5**

Prompt :

```
1 backend/core/prompts/course_v1.0.txt
```

Tu es concepteur pédagogique.

À partir des intérêts {interests} et du niveau {level}, produis :

- 3 à 5 objectifs d'apprentissage
- Un plan par modules (titres + contenus)
- 1 activité pratiques et exercices
- 1 ressource complémentaire

Style clair, concis, en français.

```
backend > core > prompts > ≡ course_v1.0.txt
1 Tu es concepteur pédagogique.
2
3 A partir des intérêts {interests} et du niveau {level}, produis :
4
5 3 à 5 objectifs d'apprentissage
6 Un plan par modules (titres + contenus)
7 1 activité pratique et 1 exercice
8 1 ressource complémentaire.
9 Style clair, concis, en français.
```

[exercice p. 4] **Solution n°6**

```
1 Endpoint /course : backend/api/main.py
```

Étape 1 : Importer les modèles Pydantic : CourseResponse et CourseRequest

Étape 2 : Importer la fonction generate_course.

Étape 3 : Création de l'endpoint

```
1 @app.post("/course", response_model=CourseResponse)
2 async def course_endpoint(req: CourseRequest):
3     text = generate_course(req.interests)
4     return CourseResponse(newsletter=text)
```

```
@app.post("/course", response_model=CourseResponse)
async def course_endpoint(req: CourseRequest):
    text = generate_course(req.interests)
    return CourseResponse(newsletter=text)
```

[exercice p. 4] **Solution n°7**

Plusieurs facteurs peuvent expliquer la latence observée. Toutefois, la cause principale reste l'utilisation d'un modèle local dont les ressources sont limitées lorsque plusieurs apprenants sollicitent le service en simultané.

Pour pallier ce problème, nous mettons en place un switch to API. Grâce à une fonction toggle et des variables d'environnement, le système pourra basculer automatiquement du modèle local vers une API externe. Ici, nous retenons l'API OpenAI car elle est performante et capable d'absorber une forte charge.

Cette intégration permettra de réduire de manière significative le temps de réponse et de limiter la latence.